

On the SUP-INF Method for Proving Presburger Formulas

ROBERT E. SHOSTAK

Stanford Research Institute, Menlo Park, California

ABSTRACT This article presents an improved version of Bledsoe's SUP-INF method for proving theorems in a subclass of Presburger arithmetic. The improved method is able to determine invalidity as well as validity, and provides counterexamples for formulas determined to be invalid. A proof of correctness is given for the algorithms on which the method is based. Implementation results are discussed, as is an application to linear programming.

KEY WORDS AND PHRASES theorem proving, program verification, deduction, Presburger arithmetic, automatic theorem proving, SUP-INF, linear programming

CR CATEGORIES 5.21, 5.41

1. Introduction

In his paper, "The SUP-INF Method in Presburger Arithmetic" [1, 2], Bledsoe presents a method for proving certain theorems in a class of logical formulas that arise frequently in program verification—Presburger formulas with universally quantified variables. At the heart of the method are algorithms (SUP and INF) intended to calculate the maximum and minimum values, respectively, that a real variable can take subject to a set of linear constraints. It is proved in that paper that the two algorithms are conservative in the sense that SUP never computes a value too small and INF never computes a value too large. That the two algorithms are conservative is sufficient to guarantee that the SUP-INF method is consistent, i.e. that a formula determined by the method to be valid is indeed valid. In this article it is shown that SUP and INF are not only conservative, but *exact* in determining the maximum and minimum real values a variable can take subject to a set of satisfiable constraints. This result gives rise to an improved method of applying the SUP and INF algorithms that is able to determine invalidity as well as validity, and that automatically finds counterexamples for formulas determined to be invalid.

The first few sections of this paper describe the original method and discuss its completeness. The next sections introduce the improved method and describe its relation to other methods and to linear programming. Implementation results are described as well. The last sections give a proof of correctness of the SUP and INF algorithms, justifying the new method.

2. The SUP-INF Method

Bledsoe's SUP-INF method operates on a subclass of formulas in Presburger arithmetic. Roughly speaking, Presburger formulas are those that can be built up from integers, integer variables, addition, multiplication by constants, the usual arithmetical relations, and the first-order logical connectives. The formula $(\forall x)(\exists y) 3x + y = 2 \supset x < y$, for example, falls within this class.

The class was first shown to be decidable by Presburger [9] in 1929. The most widely

Copyright © 1977, Association for Computing Machinery, Inc. General permission to republish, but not for profit, all or part of this material is granted provided that ACM's copyright notice is given and that reference is made to the publication, to its date of issue, and to the fact that reprinting privileges were granted by permission of the Association for Computing Machinery.

Author's address: Stanford Research Institute, Menlo Park, CA 94025

known decision algorithm for it (described by Kreisel and Krevine [6], among others) is based on a method of elimination of quantifiers. In its raw form, the algorithm tends to produce a tremendous combinatorial expansion, and is therefore not practical for nontrivial problems. More efficient versions of the method of elimination have since been given by Cooper [3, 4]. Cooper's latest algorithm [4] is the most efficient known procedure for the full theory. Recent studies by Oppen [8], in fact, have suggested that Cooper's algorithm is the best that one can possibly do in the worst case (time complexity on the order of $2^{2^{2^n}}$ in the length of the formula). Fortunately, formulas that exhibit worst-case behavior do not seem to arise often in practical contexts.

Bledsoe's SUP-INF method is intended for the subclass of Presburger formulas that, when placed in prenex form, contain only universal quantifiers. The decision complexity of this subclass is bounded above by 2^N , making it substantially easier to decide, in theory, than full Presburger.¹ Moreover, although the class of universal Presburger formulas is admittedly small in comparison with the full class, it turns out to include a remarkable proportion of the formulas (or parts of formulas) one tends to encounter in program verification and related applications.

The SUP-INF method is carried out in two stages. In the first stage the formula F to be proved is reduced to a set of integer linear programming problems (ILPs) with the property that F is valid if and only if none of the problems has a solution. In the second stage the ILPs are tested (one by one) for solvability.

The reduction to the set of ILPs consists of expanding the negation of F into a disjunctive normal form:

$$\neg F \equiv G_1 \vee G_2 \vee \dots \vee G_p,$$

where each G_i is a conjunction of linear inequalities of the form $A \leq B$. During the expansion, terms of the form $A = B$ are replaced by $(A \leq B \wedge B \leq A)$. Similarly, $A \geq B$ is replaced by $B \leq A$, $A < B$ is replaced by $A + 1 \leq B$, $\neg(A \leq B)$ is replaced by $B + 1 \leq A$, and so on. The conjunctions $G_1, G_2, \dots, G_p$ in the expanded form comprise the set of ILPs. Suppose, for example, that

$$F \equiv (x < 3y + 2) \wedge x = 1 \supset x = y.$$

Then

$$\begin{aligned} \neg F &\equiv \neg [x < 3y + 2 \wedge x = 1 \supset x = y] \\ &\equiv x < 3y + 2 \wedge x = 1 \wedge \neg(x = y) \\ &\equiv x \leq 3y + 1 \wedge x \leq 1 \wedge x \geq 1 \wedge (x + 1 \leq y \vee y + 1 \leq x) \\ &\equiv (x \leq 3y + 1 \wedge x \leq 1 \wedge x \geq 1 \wedge x + 1 \leq y) \\ &\quad \vee (x \leq 3y + 1 \wedge x \leq 1 \wedge x \geq 1 \wedge y + 1 \leq x), \end{aligned}$$

so that

$$G_1 = \{x \leq 3y + 1, x \leq 1, x \geq 1, x + 1 \leq y\}$$

and

$$G_2 = \{x \leq 3y + 1, x \leq 1, x \geq 1, y + 1 \leq x\}.$$

$\neg F$ is satisfiable if and only if either G_1 or G_2 has a solution in integers, and so F is valid if and only if neither G_1 nor G_2 has such solutions.

The second part of the SUP-INF method is concerned with testing each of the problems G_i for solvability. The testing involves the use of two recursive algorithms, SUP and INF. For a set of inequalities S and a variable v occurring in S , SUP computes the maximum value v can take in real solutions of S , and INF computes the minimum value. For example, if $S = \{0 \leq y, x \leq y, x \leq 1 - y\}$, then $\text{SUP}_S(x) = \frac{1}{2}$, $\text{INF}_S(x) = -\infty$, $\text{SUP}_S(y) = \infty$, $\text{INF}_S(y) = 0$.

¹ In a personal communication, Derek Oppen has noted that existential Presburger, the complement problem, is NP-complete

The two algorithms are used to test whether or not a set S has integer solutions as follows. First, each inequality in S that contains no variables (e.g. $2 \leq 3$) is evaluated. If one is found to be false, it can be concluded immediately that S is infeasible. If all are found to be true and if S has no inequalities containing variables, it can be concluded that S is feasible. Otherwise, for each variable x occurring in S , $\text{SUP}_S(x)$ and $\text{INF}_S(x)$ are computed. If for some x it is found that the closed interval $[\text{INF}_S(x), \text{SUP}_S(x)]$ contains no integers, S has no integer solutions. Otherwise (as will be discussed in Section 3) no conclusions can be drawn.

For example, if $S = \{1 \leq 3x, 0 \leq y, x \leq y, x \leq 1 - y\}$, we have $\text{SUP}_S(x) = \frac{1}{3}$, $\text{INF}_S(x) = \frac{1}{3}$. Since the closed interval $[\frac{1}{3}, \frac{1}{3}]$ contains no integers, S must have no integer solutions.

The definitions of SUP and INF we have given here (Figures 1, 2, 3) are modified (though equivalent) versions of the originals. The chief difference is that the new versions assume that their arguments have been canonicalized using the SIMP routine² described in [1]. We have implemented the new versions, and have in fact found them to be more efficient than the originals.

The definition of INFF (an auxiliary function called by INF) has also been changed, so as to be perfectly symmetrical with its counterpart SUPP. The asymmetry in the original versions reflects an implicit constraint requiring all variables to take only nonnegative values in solutions of S . We have removed the constraint so as to make SUP and INF perfect duals and to make certain definitions cleaner. Of course, one can reincorporate the restriction by adding new inequalities of the form $0 \leq x$ to S .

As can be seen from their definitions, SUP and INF actually take two arguments rather than one. The first formal argument J is bound to the variable whose maximum (or minimum) value is to be computed; the second argument H is bound to the empty set $\emptyset$. On internal recursive calls, J is bound to expressions that may involve $+$, $*$, MAX, and MIN, and H is bound to sets of variables occurring in S .

In addition to the auxiliary functions SUPP and INFF, SUP and INF make use of functions $\text{UPPER}_S(x)$ and $\text{LOWER}_S(x)$ defined as follows: Let x be a variable occurring in S and let $\{x \leq U_1, x \leq U_2, \dots, x \leq U_n, L_1 \leq x, L_2 \leq x, \dots, L_m \leq x\}$ be obtained from S by solving each inequality in S that mentions x for x . Then

$$\text{UPPER}_S(x) = \begin{cases} \text{MIN}(U_1, U_2, \dots, U_n) & \text{if } n > 1, \\ U_1 & \text{if } n = 1, \\ \infty & \text{else,} \end{cases}$$

and

$$\text{LOWER}_S(x) = \begin{cases} \text{MAX}(L_1, L_2, \dots, L_m) & \text{if } m > 1, \\ L_1 & \text{if } m = 1, \\ -\infty & \text{else.} \end{cases}$$

Returning to the example $S = \{0 \leq y, x \leq y, x \leq 1 - y\}$, we have

$$\begin{aligned} \text{UPPER}_S(x) &= \text{MIN}(y, 1 - y), & \text{LOWER}_S(x) &= -\infty, \\ \text{UPPER}_S(y) &= 1 - x, & \text{LOWER}_S(y) &= \text{MAX}(0, x). \end{aligned}$$

Then

$$\begin{aligned} \text{SUP}_S(x, \emptyset) &= \\ &= \text{SUPP}(x, \text{SIMP}(\text{SUP}_S(\text{UPPER}_S(x), \{x\}))) && (\text{step 2.2}) \\ &= \text{SUPP}(x, \text{SIMP}(\text{SUP}_S(\text{MIN}(y, 1 - y), \{x\}))) \\ &= \text{SUPP}(x, \text{SIMP}(\text{MIN}(\text{SUP}_S(y, \{x\}), \text{SUP}_S(1 - y, \{x\})))) && (\text{step 5}) \\ &\quad \text{SUP}_S(y, \{x\}) = \\ &\quad \text{SUPP}(y, \text{SIMP}(\text{SUP}_S(\text{UPPER}_S(y), \{x, y\}))) && (\text{step 2.2}) \\ &\quad = \text{SUPP}(y, \text{SIMP}(\text{SUP}_S(1 - x, \{x, y\}))) \\ &\quad \quad \text{SUP}_S(1 - x, \{x, y\}) = \\ &\quad \quad \text{Let } B' = \text{SUP}_S(1, \{x, y\}) \\ &\quad \quad = 1 && (\text{step 1}) \end{aligned}$$

² In essence, the SIMP routine distributes $*$ over $+$ and pulls MAX's and MIN's out to the front. The SIMPed version of $4 * (3 + \text{MAX}(2, X))$, for example, is $\text{MAX}(20, 12 + 4 * A)$.

$$\begin{aligned}
&= \text{SUP}_S(-x, \{x, y\}) + 1 && \text{(step 4 2)} \\
&= -\text{INF}_S(x, \{x, y\}) + 1 && \text{(step 3.1)} \\
&= -x + 1 && \text{(step 2.1)} \\
&= \text{SUPP}(y, -x + 1) \\
&= -x + 1 && \text{(step 4.2 in SUPP)} \\
\text{SUP}_S(1 - y, \{x\}) &= \\
&\text{Let } B' = \text{SUP}_S(1, \{x, y\}) \\
&= 1 && \text{(step 1)} \\
&= \text{SUP}_S(-y, \{x\}) + 1 && \text{(step 4 2)} \\
&= -\text{INF}_S(y, \{x\}) + 1 && \text{(step 3 1)} \\
&\text{INF}_S(y, \{x\}) = \\
&\quad \text{INFF}(y, \text{SIMP}(\text{INF}_S(\text{LOWER}_S(y), \{x\}))) && \text{(step 2 2)} \\
&= \text{INFF}(y, \text{SIMP}(\text{INF}_S(\text{MAX}(0, x), \{x\}))) \\
&\quad \text{INF}_S(\text{MAX}(0, x), \{x\}) = \\
&\quad \text{MAX}(\text{INF}_S(0, \{x\}), \text{INF}_S(x, \{x\})) \\
&= \text{MAX}(0, \text{INF}_S(x, \{x\})) && \text{(step 1)} \\
&= \text{MAX}(0, x) && \text{(step 2 1)} \\
&= \text{INFF}(y, \text{MAX}(0, x)) \\
&= \text{MAX}(\text{INFF}(y, 0), \text{INFF}(y, x)) \\
&= \text{MAX}(0, \text{INFF}(y, x)) && \text{(step 1 in INFF)} \\
&= \text{MAX}(0, x) && \text{(step 4 2 in INFF)} \\
&= \text{SUPP}(x, \text{SIMP}(\text{MIN}(-x + 1, -(\text{MAX}(0, x)) + 1))) \\
&= \text{SUPP}(x, \text{MIN}(1 - x, 1, 1 + x)) \\
&= \text{MIN}(\text{SUPP}(x, 1 - x), \text{SUPP}(x, 1), \text{SUPP}(x, 1 + x)) && \text{(step 3 in SUPP)} \\
&\quad \text{SUPP}(x, 1 - x) = \frac{1}{2} && \text{(step 4.2 in SUPP)} \\
&\quad \text{SUPP}(x, 1) = 1 && \text{(step 1 in SUPP)} \\
&\quad \text{SUPP}(x, 1 + x) = \infty && \text{(step 4 3 3 in SUPP)} \\
&= \text{MIN}(\frac{1}{2}, 1, \infty) \\
&= \frac{1}{2}
\end{aligned}$$

It might be noted that the internal calls to SUP and INF in the example sometimes provide arguments J other than variables and arguments H other than the empty set. Note also that the two algorithms sometimes return values other than numbers. In general, SUP and INF return expressions involving the variables in H . Note, for example, that $\text{SUP}_S(y, \{x\}) = -x + 1$. We show later on (Lemma 15) that if J is a variable and H is an arbitrary set of variables, the expression E returned by $\text{SUP}_S(J, H)$ can be interpreted as follows: If real numerical values are assigned to the variables in H , and E is evaluated for those values, the result is the maximum real value that J can have in solutions of S when the variables in H are fixed at their assigned values. For example, letting $J = y$ and $H = \{x\}$, we have $E = \text{SUP}_S(y, \{x\}) = -x + 1$. If we assign -2 to x , E becomes 3 . Thus, 3 is the maximum value y can have in solutions of S when x is fixed at -2 . The dual remarks, of course, apply in INF.

3. Completeness

In his original paper describing the SUP-INF method, Bledsoe proved that the SUP and INF algorithms are conservative in the sense that $\text{SUP}_S(x, \emptyset)$ always computes a number greater than or equal to the maximum real value x can have in solutions of S , and $\text{INF}_S(x, \emptyset)$ always computes a value less than or equal to the minimum real value. These properties are together sufficient to prove that the SUP-INF method is sound—that is, if for some variable x , $[\text{INF}_S(x), \text{SUP}_S(x)]$ contains no integers, the original formula must indeed be valid.

If, on the other hand, the interval $[\text{INF}_S(x), \text{SUP}_S(x)]$ does contain an integer for each variable x occurring in S , the original formula F may or may not be valid. Consider, for example, the two formulas

$$F_1 = \forall x \forall y \neg (2x + 3y = 1) \quad \text{and} \quad F_2 = \forall x \forall y \neg (2x + 2y = 1).$$

The formula F_1 is clearly not valid since if $x = -1$ and $y = 1$, the expression $\neg (2x + 3y = 1)$ evaluates to false. F_2 , however, is valid. To see this, just note that the left-hand side of the equality $2x + 2y = 1$ must always be even, while the right-hand side is odd.

Algorithm $SUP_s(J, H)$

IF	Action	Return
1 J is a number		J
2 J is a variable		J
2.1 $J \in H$		
2.2 $J \notin H$	Let $Q = UPPER_s(J)$ $Z = SUP_s(Q, H \cup \{J\})$	$SUPP(J, SIMP(Z))$
3. $J = rA$ where r is a number		
3.1 $r < 0$		$r \text{ INF}_s(A, H)$
3.2 $r \geq 0$		$r \text{ SUP}_s(A, H)$
4 $J = rv + B$ where r is a number, v a variable	Let $B' = SUP_s(B, H \cup \{v\})$ Let $J' = SIMP(rv + B')$	$SUP_s(J', H)$ $SUP_s(rv, H) + B'$
4.1 v occurs in B'		
4.2 v does not occur in B'		
5 $J = \text{MIN}(A, B)$		$\text{MIN}(SUP_s(A, H), SUP_s(B, H))$

FIG 1

Algorithm $INF_s(J, H)$

IF	Action	Return
1 J is a number		J
2 J is a variable		J
2.1 $J \in H$		
2.2 $J \notin H$	Let $Q = LOWER_s(J)$ $Z = INF_s(Q, H \cup \{J\})$	$INFF(J, SIMP(Z))$
3. $J = rA$ where r is a number		
3.1 $r < 0$		$r \text{ SUP}_s(A, H)$
3.2 $r \geq 0$		$r \text{ INF}_s(A, H)$
4. $J = rv + B$ where r is a number, v a variable	Let $B' = INF_s(B, H \cup \{v\})$ Let $J' = SIMP(rv + B')$	$INF_s(J', H)$ $INF_s(rv, H) + B'$
4.1 v occurs in B'		
4.2 v does not occur in B'		
5 $J = \text{MAX}(A, B)$		$\text{MAX}(INF_s(A, H), INF_s(B, H))$

FIG 2

Algorithm $SUPP(x, y)$

IF	Return
1 y is a number	y
2 $x = y$	∞
3 $y = \text{MIN}(A, B)$	$\text{MIN}(SUPP(x, A), SUPP(x, B))$
4. $y = bx + c$ where x does not occur in c	
4.1 $b > 1$	∞
4.2 $b < 1$	$c/1 - b$
4.3 $b = 1$	
4.3.1 c not a number	∞
4.3.2 $c < 0$	$-\infty$
4.3.3 $c \geq 0$	∞

Algorithm $INFF(x, y)$

IF	Return
1 y is a number	y
2 $x = y$	$-\infty$
3. $y = \text{MAX}(A, B)$	$\text{MAX}(INFF(x, A), INFF(x, B))$
4 $y = bx + c$ where x does not occur in c	
4.1 $b > 1$	$-\infty$
4.2 $b < 1$	$c/1 - b$
4.3 $b = 1$	
4.3.1 c not a number	$-\infty$
4.3.2 $c > 0$	∞
4.3.3 $c \leq 0$	$-\infty$

FIG 3

Reducing F_1 and F_2 to sets of inequalities in accordance with the method, one obtains the single set $S_1 = \{2x + 3y \leq 1, 1 \leq 2x + 3y\}$ from F_1 , and the single set $S_2 = \{2x + 2y \leq 1, 1 \leq 2x + 2y\}$ from F_2 . Applying the SUP and INF algorithms gives:

$$\begin{aligned}\text{INF}_{S_1}(x, \emptyset) &= \text{INF}_{S_2}(x, \emptyset) = \text{INF}_{S_1}(y, \emptyset) = \text{INF}_{S_2}(y, \emptyset) = -\infty, \\ \text{SUP}_{S_1}(x, \emptyset) &= \text{SUP}_{S_2}(x, \emptyset) = \text{SUP}_{S_1}(y, \emptyset) = \text{SUP}_{S_2}(y, \emptyset) = \infty.\end{aligned}$$

Thus, the intervals $[\text{INF}_{S_1}(x, \emptyset), \text{SUP}_{S_1}(x, \emptyset)]$, $[\text{INF}_{S_1}(y, \emptyset), \text{SUP}_{S_1}(y, \emptyset)]$, $[\text{INF}_{S_2}(y, \emptyset), \text{SUP}_{S_2}(y, \emptyset)]$, and $[\text{INF}_{S_2}(x, \emptyset), \text{SUP}_{S_2}(x, \emptyset)]$ all contain integers, though F_2 is valid and F_1 is not.

The difficulty lies in the fact that the application of SUP and INF as described above is not by itself sufficient to test whether or not an integer linear programming problem has solutions. As we will prove, SUP and INF do find the maximum and minimum *real* values a variable can have subject to a set of linear constraints. (Note, for example, that x may indeed have arbitrarily large or small real values in solutions of S_1 and S_2 .) However, even though there may be an integer value x_0 between the two extreme real values of some variable x , there may be no integer values $y_0, z_0, \dots$ of the other variables $y, z, \dots$ for which the point $(x_0, y_0, z_0, \dots)$ satisfies the constraints. Such is the case for S_2 .

Fortunately this incompleteness manifests itself only rarely in practice. The reasons are twofold. First, the validity of programs encountered in practice almost never depends on the sort of Diophantine behavior typified by the above example. Second, the method used to reduce formulas to sets of linear inequalities tends to produce sets that have integer solutions if and only if they have real solutions. For example, the valid formula $F \equiv 3x \geq 10 \supset x \geq 4$ gives rise to $S = \{3x \geq 10, x \leq 3\}$. S has no solution in reals even though F is not valid in reals. (Note that in obtaining S from F , the expression $\neg(x \leq 3)$ is replaced by $4 \leq x$; this transformation clearly depends on the integrality of x .)

Unfortunately there is no simple syntactic characterization of the class of formulas that evince the incompleteness. If the method fails to find a variable x for which $[\text{INF}(x, \emptyset), \text{SUP}(x, \emptyset)]$ contains no integers, one cannot easily tell by examining the given formula whether or not it can be concluded that the formula is invalid. The SUP-INF method, in its original form, is therefore only a semidecision procedure; it can demonstrate validity, but not invalidity.

Section 4 describes an improved method that remedies this inadequacy. The new method is also incomplete, but is able to identify formulas that fall outside the domain of completeness. It can therefore prove both validity and invalidity.

4. The Improved Method

As described in Section 1, the original method for testing whether or not a given set S of inequalities (each of which contains at least one variable) has solutions is as follows: A variable v occurring in S is chosen at random, and $\text{SUP}_S(v, \emptyset)$ and $\text{INF}_S(v, \emptyset)$ are computed. If the closed interval $[\text{INF}_S(v, \emptyset), \text{SUP}_S(v, \emptyset)]$ has no integer values, the procedure halts, with the conclusion that S has no integer solutions. Otherwise, a new variable is chosen, and the procedure is repeated. If all of the variables occurring in S have been exhausted, the method fails.³

We propose here a revised method of applying SUP and INF that has a number of advantages over the original. First, the new method is complete over a somewhat larger domain than the original. Much more important, the new method provides a full decision procedure over its domain of completeness rather than just a semidecision procedure. Finally, and perhaps most important, the new method produces counterexamples for formulas that are found to be invalid.

³ The case in which S contains exactly one variable (say v) is exceptional. In this case S is feasible if and only if $[\text{INF}_S(v, \emptyset), \text{SUP}_S(v, \emptyset)]$ contains an integer.

The method proceeds by eliminating variables from S one by one until a set of inequalities is obtained that is either clearly unsatisfiable, or that has no variables occurring. (A set having no variables can, of course, be tested easily by testing each inequality in the set.) At each step, a variable v is chosen and $\text{SUP}_S(v, \emptyset)$ and $\text{INF}_S(v, \emptyset)$ are computed. If $\text{INF}_S(v, \emptyset) > \text{SUP}_S(v, \emptyset)$, it follows from the consistency of the original method that S is unfeasible. Otherwise an integer value in between is chosen (or a real, if there is no integer in the interval) and substituted for v in all inequalities of S . It follows from the main theorem to be proved in Section 12 that the new set is satisfiable in reals if the old one was, and so the process can be repeated.

If unsatisfiability has not been detected by the time all variables have been eliminated, the set of substituted values provides a real solution to S . In most instances, the real solution is also an integer solution, hence a counterexample to the given Presburger formula.

For example, the formula $(3x \geq y \geq 2) \supset (x + y > 3)$ leads to the set $S = \{3x \geq y, y \geq 2, x + y \leq 3\}$. Selecting x to SUP on first, one obtains $\text{SUP}(x, \emptyset) = 1$, $\text{INF}(x, \emptyset) = \frac{2}{3}$. The integer value 1 is chosen from the interval $[\frac{2}{3}, 1]$ and substituted for x in S , giving $\{3 \geq y, y \geq 2, y \leq 2\}$. SUPing on y then gives $\text{SUP}(y, \emptyset) = 2$, $\text{INF}(y, \emptyset) = 2$. The value 2 is chosen from the interval $[2, 2]$ and used for y , giving the set $\{3 \geq 2, 2 \geq 2, 1 + 2 \leq 3\}$. This latter is easily seen to be satisfiable; hence $x = 1, y = 2$ must provide a counterexample for the given formula.

Consider, as a second example, the formula

$$(y + z < 5 \wedge 3z \geq y + 8 \wedge x \leq y) \supset (2 + z > 2y \vee x > z - 5).$$

Applying the method described earlier, one obtains the set

$$S = \{y + z \leq 4, 3z \geq y + 8, x \leq y, 2 + z \leq 2y, x \leq z - 5\}.$$

Selecting x to SUP on first gives $\text{SUP}(x, \emptyset) = -3$, $\text{INF}(x, \emptyset) = -\infty$. Choosing the value $x = -3$ from the interval $[-\infty, -3]$ and substituting gives $\{y + z \leq 4, 3z \geq y + 8, -3 \leq y, 2 + z \leq 2y, -3 \leq z - 5\}$. Next SUPing on y , one obtains $\text{SUP}(y, \emptyset) = 1$, $\text{INF}(y, \emptyset) = \frac{1}{3}$. Since $\text{SUP}(y, \emptyset) < \text{INF}(y, \emptyset)$, S must have no real solution, hence no integer solution. The given formula is therefore valid.

In certain instances the algorithm will return a real solution in which one or more variables have noninteger values. If, in particular, the first variable (say x) selected for SUPing has a noninteger value, then it must have been that the interval $[\text{INF}(x, \emptyset), \text{SUP}(x, \emptyset)]$ contained no integers. In this case, S has real solutions but no integer solutions.

If on the other hand the first variable selected has an integer value, but some other variable in the solution has a noninteger value, S may or may not have an integer solution and so, in this case, the method is inconclusive. Consider, for example, the set $S = \{3x + 5y \leq 7, 3x + 5y \geq 7\}$. Selecting x first gives $\text{SUP}(x, \emptyset) = \infty$, $\text{INF}(x, \emptyset) = -\infty$. Now if 0 is chosen from the interval $[-\infty, \infty]$ and substituted for x , the noninteger solution $x = 0, y = \frac{7}{5}$ is found. The set S does, however, have integer solutions, e.g. $x = -1, y = 2$. In this sort of situation, the new method suffers from the same sort of incompleteness as the old.

Completeness can be obtained, of course, by substituting, in turn, all integer values in the closed interval rather than simply choosing one. Since each variable will, in general, lead to a number (possibly an infinite number) of such substitutions, the search for a solution assumes the form of a tree. This idea is further refined in the branch and bound methods of integer linear programming. The incompleteness of the basic method is of so little practical consequence, however, that schemes for augmenting the method in order to obtain completeness are largely a matter of theoretical interest.

5. Comparison with Cooper's Method

The SUP-INF approach may be viewed as transforming an integer problem into the

real domain, solving it in that domain, and then interpreting the result back in the integer domain. The success of the method derives at least in part from the fact that the real problem is easier to solve than the integer problem. The price paid for this ease of solution is, of course, the resulting incompleteness.

The SUP-INF method's dependence on the properties of reals distinguishes it from Cooper's method, which operates completely within the integer domain. In comparing the two, one must remember that Cooper's method operates over the full Presburger class, rather than just the subclass of universally quantified formulas.

The author is not aware of any thorough theoretical or empirical analyses of the relative performance of the two methods. One can show fairly easily that the SUP-INF method, both in its original and revised forms, is of deterministic complexity on the order of 2^n in the lengths of the formula. We noted earlier that Cooper's method operates with complexity on the order of 2^{2^n} over the full Presburger class. The complexity manifests itself as a combinatorial expansion in the lengths of the formula as the method proceeds. A large component of this expansion is associated with alternation of quantifiers, which does not occur in the universal subclass. For this reason, Cooper's method can be expected to perform with greater efficiency over the subclass than it does over the full class.

However, a second component of the expansion—one that depends on the least common multiple of the coefficients of variables appearing in the formula—does manifest itself in the subclass. This "delta" expansion can be mitigated in certain cases using a technique Cooper describes in [4]. In cases where the delta expansion cannot be treated, one would expect the SUP-INF method, which is not subject to the expansion, to perform more efficiently.

6. Implementation Results

The improved SUP-INF method has been implemented in INTERLISP for the PDP-10 computer and used extensively in proving verification conditions arising from JOVIAL programs. The prover does not require any special initial form, and handles simple examples in a second or less of CPU time. Most larger examples are solved many times faster than a human could solve them. The counterexample feature has proved quite useful in locating bugs in programs submitted to the verification process.

We have found it efficient to implement the substitution of values by adding another argument to SUP and INF which acts as an A-list retaining substituted values. The functions UPPER(v) and LOWER(v) are modified to test whether v is bound on the A-list and to return the bound value if it is.

7. Relation to Linear Programming

As we prove in the next sections, $\text{SUP}_S(v, \emptyset)$ computes the maximum real value a variable v can have subject to a set of S of linear constraints. It is easy to see that SUP can be applied to arbitrary real linear programming problems. If a linear objective function L is to be maximized subject to a set S of constraints, the inequality $x \leq L$ is added to S , where x is a new variable. Clearly, $\max_S L = \max_S x$, and so $\text{SUP}(x, \emptyset)$ provides the desired answer. If L is to be minimized, of course, $L \leq x$ is added and $\text{INF}(x, \emptyset)$ is computed.

Conversely, one could ask whether a well-known real linear programming algorithm might make a good substitute for SUP and INF in the method—or whether a known integer linear programming method could replace the second part of the method. In order to answer these questions, the author coded (in INTERLISP) both the Simplex algorithm for real linear programming and the Gomory cutting plane algorithm for integer linear programming. Surprisingly, neither of these implementations performed more efficiently than the SUP-INF implementation on naturally arising formulas. The rather poor relative performance of the well-known algorithms stems from the fact that typical problems tend to be small (thus emphasizing the overhead necessary to convert

the problem to matrix form) and structured in such a way as to produce sparse matrices. The Gomory algorithm, moreover, could not be trusted to terminate on unbounded problems.

8. Preliminary Remarks About the Main Theorem

In the original paper on the SUP-INF method, it was shown that if S is a satisfiable set of linear inequalities and if Q is an arbitrary quasilinear⁴ expression, $\max_S Q \leq \text{SUP}_S(Q, \emptyset)$. In other words, it was shown that $\text{SUP}_S(Q, \emptyset)$ always returns a value at least as great as the least upper bound of the values Q can take in solutions of S . Experience with the algorithm, moreover, suggested that if the set S is in "natural form" (i.e. if UPPER is defined as described earlier) then the inequality is exact—i.e. $\max_S Q = \text{SUP}_S(Q, \emptyset)$. In fact this turns out not to be the case for arbitrary quasilinear forms Q , or even for arbitrary linear forms Q . For example if $S = \{x \leq y, x \leq 2 - y\}$ and $Q = y + 2x$, then $\max_S Q = 3$ whereas $\text{SUP}_S(Q, \emptyset) = \infty$. In order to prove the correctness of the improved method, however, it suffices only to show that if v is a *variable*, then $\max_S v = \text{SUP}_S(v, \emptyset)$.

Now it follows from Bledsoe's result, $\max_S v \leq \text{SUP}_S(v, \emptyset)$, that if v is unbounded (can have arbitrarily large values) in solutions of S then $\infty = \max_S v = \text{SUP}_S(v, \emptyset)$. Thus, it will be sufficient to consider only the case in which v is bounded in S , so that $\max_S v$ is finite.

The proof requires a number of elementary results in linear algebra. The next few sections will be concerned with developing the definitions and theorems needed to support the main argument.

9. Preliminary Definitions

Throughout our discussion we will assume that V is a finite set of variables.

Definition. A *linear form* in V is an expression of the form $r_1 v_1 + r_2 v_2 + \cdots + r_n v_n + c$, where $n \geq 0$, each $v_i \in V$, each r_i is a nonzero real, and c is a real.

For convenience, we will not bother writing down unity coefficients and zero trailing constants. Thus, we will write $x + 2y$ in place of $1x + 2y + 0$.

Definition. A *minilinear form* in V is either a linear form in V , an expression of the form $\text{MIN}(L_1, L_2, \dots, L_n)$ where $n \geq 1$ and each L_i is linear in V , or one of $\infty, -\infty$.

In the sequel we assume all linear (minilinear) forms to be linear (minilinear) in the set V .

Definition. An *inequality* is an expression of the form $A \leq B$, where A and B are linear forms having no variables in common.

Definition. A *point* (with respect to V) is an assignment of reals to the members of V . If P is a point and Q is a minilinear form, the *value of Q at P* , written $Q(P)$, is the real obtained by evaluating Q in the customary way with each variable replaced by its assignment in P .

Definition. A point P *satisfies an inequality* $A \leq B$ if $A(P) \leq B(P)$ and *satisfies a set S of inequalities* if it satisfies each member of S . S is said to be *satisfiable* if some point satisfies S and *unsatisfiable* otherwise.

Definition. If r is a real, Q is minilinear, and S is a set of inequalities, we say that Q *can have the value r in S* if there exists a point P satisfying S such that $Q(P) = r$. We say that Q *has the unique value r in S* if Q can have the value r in S , but no other value.

Finally, we define the $\max_S$ function as follows:

Definition.

$$\max_S Q = \begin{cases} r & \text{if } Q \text{ can have the real value } r \text{ in } S \\ & \text{but no greater value,} \\ \text{undefined} & \text{otherwise.} \end{cases}$$

⁴ Quasilinear expressions are linear expressions that may have occurrences of MAX or MIN, e.g. $3 + \text{MAX}(2y - \text{MIN}(4Z, 3), x + y)$

These definitions immediately give rise to the following theorem:

THEOREM 1. *Let $L_1, L_2, \dots, L_n$ be linear forms having unique values in S . Then*

$$\max_S[\text{MIN}(L_1, \dots, L_n)] = \text{MIN}(\max_S L_1, \dots, \max_S L_n).$$

PROOF. Let the unique value of each L_i be r_i . Then

$$\max_S[\text{MIN}(L_1, \dots, L_n)] = \text{MIN}(r_1, \dots, r_n) = \text{MIN}(\max_S L_1, \dots, \max_S L_n). \quad \square$$

10 Results on Minimally Bounding Sets

The first few results given here can be found in books on convex linear algebra. We will omit proofs for some of the more obvious of these.

It will be convenient to think of the points defined in Section 9 as points in the Euclidean space R^n of dimension n , where n is the size of the set V . Conventional vector notation will be used.

Definition. A set C of points is said to be *convex* if

$$\left. \begin{array}{l} P_1 \in C \\ P_2 \in C \\ 0 \leq t \leq 1 \end{array} \right\} \rightarrow tP_1 + (1-t)P_2 \in C.$$

Thus, C is convex if for each two points P_1, P_2 in C , all points on the line segment P_1P_2 are in C .

THEOREM 2. *The set of points satisfying in inequality $A \leq B$ is convex.*

THEOREM 3. *The intersection of convex sets is again convex.*

THEOREM 4. *The points satisfying a set of inequalities form a convex set.*

PROOF. From Theorems 2 and 3.

Definition. Suppose S is a set of inequalities in V , R is a linear form in V , and r is a real. Then S *bounds* R at r if $\max_S R$ is defined and equal to r .

For example, $S = \{x \leq y, x \leq 2 - y\}$ bounds $R = x + y$ at 2

Definition. S *bounds* R if for some real r , S bounds R at r

Definition. S *minimally bounds* R at r if S bounds R at r and no proper subset of S bounds R at r .

Definition. S *minimally bounds* R if S bounds R and no proper subset of S bounds R .

It is clear from the definitions that if S minimally bounds R , then S minimally bounds R at r , for some r (namely, at $r = \max_S R$). It is not so apparent, however, that the converse holds. One might imagine a situation in which S minimally bounds R at r but in which some proper subset of S bounds R at a value greater than r . As we will see, this cannot happen. It will turn out that if S minimally bounds R at r , S minimally bounds R . Theorem 5 will take us a long way in establishing this fact.

THEOREM 5. *Let S minimally bound R at r , and let S_E denote the set of equalities obtained from S by replacing $\leq$ with $=$. Then any point P satisfying S such that $R(P) = r$ satisfies S_E .*

PROOF. Suppose P satisfies S with $R(P) = r$ but that for some inequality $A \leq B$ in S , $A(P) \neq B(P)$. Since P satisfies S , then $A(P) < B(P)$. Now obtain S' from S by removing $A \leq B$. By the minimality of S , there exists a point P' satisfying S' such that $R(P') > r$. Now consider the line segment PP' between P and P' . Both P and P' satisfy S' , so by Theorem 3 and the definition of convexity, all points on PP' satisfy S' . Now since P' cannot satisfy S , $A(P') > B(P')$. Then since $A(P) < B(P)$, it is easy to verify that there must be a point Q on PP' such that $A(Q) = B(Q)$. Since Q satisfies S' and $A(Q) = B(Q)$, Q must satisfy S . But since $R(P) = r$ and $R(P') > r$, $R(Q) > r$, giving a contradiction.

For example, again letting $S = \{x \leq y, x \leq 2 - y\}$ and $R = x + y$, we see that at each point satisfying S at which R achieves its maximum value of 2 (there is only one: $x = y = 1$), $x = y$ and $x = 2 - y$.

Now as we noted above, if S minimally bounds R , then S minimally bounds R at r where $r = \max_S R$. Theorem 5 then gives us the following corollary:

THEOREM 6. Suppose S minimally bounds R . Then any point P that satisfies S such that $R(P) = \max_S R$ also satisfies S_E .

We now have enough to prove:

THEOREM 7. If S minimally bounds R at r , S minimally bounds R .

PROOF. Let S' be any proper subset of S . We claim that R is unbounded in S' . So suppose R is bounded in S' . Let T be a smallest subset of S' in which R is bounded and suppose T bounds R at r' . Now since S minimally bounds R at r , $r' > r$. Let P be a point satisfying S such that $R(P) = r$ and P' a point satisfying T such that $R(P') = r'$. By Theorem 5, P satisfies S_E , and by Theorem 6, P' satisfies T_E . Since $T_E \subseteq S_E$, in fact, both P and P' satisfy T_E . Now it is easy to show (we leave it as an exercise) that if two distinct points satisfy a set of linear equalities, all points on the line that they determine are also solutions. Since $R(P) \neq R(P')$, R takes arbitrarily large (or small, for that matter) values on the line PP' . Thus, R can acquire arbitrarily large values at points satisfying T_E and hence T . But this is impossible since R is bounded by r' in T , and so we have a contradiction. $\square$

Theorem 7 gives us the following corollary:

THEOREM 8. Suppose S bounds R . Then there exists a subset T of S such that T minimally bounds R and $\max_S R = \max_T R$.

PROOF. Let T be a smallest subset of S that bounds R at $\max_S R$. Then $\max_S R = \max_T R$. Furthermore, by Theorem 7, T minimally bounds R . $\square$

Minimal bounding sets have two other properties that will be needed to prove the main theorem:

THEOREM 9. Suppose $v \in V$, $v \leq R$ is in S , and S minimally bounds v . Then if T is obtained from S by substituting $v_M = \max_S v$ everywhere for v , R has the unique value v_M in T .

PROOF. Since T contains $v_M \leq R$, R cannot take a value smaller than v_M in solutions of T . On the other hand, if P satisfies T and $v_M < R(P)$, P must satisfy S (with v taking the value v_M), contradicting Theorem 6. $\square$

For example, suppose $S = \{x \leq y + z, y + z \leq 3\}$. Then $\max_S x = 3$, and $T = \{3 \leq y + z, y + z \leq 3\}$. Clearly $R = y + z$ has the unique value 3 in solutions of T .

THEOREM 10. Suppose S minimally bounds v and $v \leq L_1, v \leq L_2, \dots, v \leq L_n$ are inequalities in S . Then there exists a real $v_0 < \max_S v$ such that if T is obtained from S by replacing v by v_0 , $\max_T L_i > v_0$ for $1 \leq i \leq n$.

PROOF. It will suffice to show that there exists a point P satisfying S such that $v(P) < \max_S v$ and for $1 \leq i \leq n$, $v(P) < L_i(P)$. For then, letting $v_0 = v(P)$, we will have $v_0 < \max_S v$ and $v_0 < L_i(P) \leq \max_T L_i$ for each i . So let $c > 0$ be a real, and obtain S' from S by replacing each $v \leq L_i$ by $v \leq L_i - c$. We claim that S'_E (i.e. S' considered as a set of equalities) has a solution. To see this, first note that S_E has (by Theorem 5) a solution. S therefore has no "parallel planes" except possibly pairs of inequalities of the form $A \leq B, B \leq A$. Furthermore, S can have no such pairs—for suppose $A \leq B, B \leq A$ are members of S . By minimality, there exist points P_1 satisfying $S - \{A \leq B\}$ and P_2 satisfying $S - \{B \leq A\}$ such that $v(P_1) > \max_S v$, $v(P_2) > \max_S v$. But then by convexity there is a point P_3 on the line segment in between satisfying S such that $v(P_3) > \max_S v$, giving a contradiction. Thus, the equalities in S'_E are linearly independent and so S'_E must have a solution.

Now let P be such a solution. P satisfies S' , hence S . Moreover, $v(P) = L_i(P) - c$ and so $v(P) < L_i(P)$ for each i . It remains to see that $v(P) < \max_S v$. Since P satisfies S , $v(P) \leq \max_S v$. Now if $v(P) = \max_S v$, we would have by Theorem 5 that P satisfies S_E . This cannot be, because P satisfies S'_E . Hence $v(P) < \max_S v$, as required. $\square$

11. Some Needed Properties of SUP and SUPP

In addition to the results of Section 10, the main argument of the proof requires a number of theorems about the algorithms SUP and SUPP.

Theorems 12 and 13 are straightforwardly obtained using recursion induction, and so we will omit their proofs. In each case S is assumed to be a satisfiable set of linear inequalities and A a minilinear form.

THEOREM 11 (Bledsoe). *If S is satisfiable, $SUP_S(A, \emptyset) \leq \max_S(A)$.*

THEOREM 12. *$SIMP[SUP_S(A, L)]$ is a minilinear form in L .*

THEOREM 13. *If $T \subseteq S$, $SUP_S(A, \emptyset) \leq SUP_T(A, \emptyset)$.*

THEOREM 14. *Suppose z is a minilinear form in $\{v\}$ having a fixed point at $v = v_M$ and such that for some $v' < v_M$, $Z(v') > v'$. Then $SUPP(v, Z) = v_M$.*

PROOF. Since Z is minilinear in $\{v\}$, Z can be expressed either as L_1 , or $\min(L_1, \dots, L_N)$, $N \geq 1$, where each L_i is linear in $\{v\}$. We will assume the latter. The former case is handled similarly.

By definition of $SUPP$,

$$SUPP(v, Z) = SUPP(v, \min(L_1, \dots, L_N)) = \min(SUPP(v, L_1) \dots SUPP(v, L_N)).$$

Now since $Z(v_M) = v_M$, there must be an L_i (say L_1) such that $L_1(v_M) = v_M$. Furthermore since $v' < v_M$ and $Z(v') > v'$, L_1 must have slope less than 1, and so can be expressed $b_1 v + c_1$ where $b_1 < 1$. By definition of $SUPP$, $SUPP(v, L_1) = c_1 / (1 - b_1)$. Since $v_M = b_1 v_M + c_1$, $v_M = c_1 / (1 - b_1)$ and so $SUPP(v, L_1) = v_M$. It thus suffices to show that for each i , $2 \leq i \leq N$, $SUPP(v, L_i) \geq v_M$.

Now if we express L_i as $b_i v + c_i$, the condition $z(v') > v'$ requires that either $b_i = 1$ and $c_i > 0$, or $b_i \neq 1$. In the former case $SUPP(v, L_i) = \infty \geq v_M$ as required. In the latter case either $b_i > 1$ so that once again $SUPP(v, L_i) = \infty$, or $b_i < 1$. In this last case $SUPP(v, L_i) = c_i / (1 - b_i)$.

Now since $z(v_M) = v_M$, $L_i(v_M) \geq v_M$, hence $b_i v_M + c_i \geq v_M$. $b_i < 1$ then gives that $c_i / (1 - b_i) \geq v_M$, so that $SUPP(v, L_i) \geq v_M$ as needed. $\square$

The last, and perhaps most important, theorem explicates the role of the second argument to SUP :

THEOREM 15. *Let v be a variable not occurring in A , let r be a real, and obtain T from S by substituting r everywhere for v . Then if T is satisfiable, $SUP_T(A, \emptyset)$ equals $SUP_S(A, \{v\})$ evaluated at $v = r$.*

PROOF. We need to generalize the theorem so that it will carry itself through induction. To this end, we introduce the following notation: If L is a set of variables, and $\sigma: L \rightarrow \text{Reals}$ is a substitution, we let $S\sigma$ and $A\sigma$ denote, respectively, the results of applying σ to S and to A .

The generalized theorem is:

THEOREM 16. *Say L, L' are sets of variables, $L' \subseteq L$, $\sigma: L - L' \rightarrow \text{Reals}$, A is minilinear, and $SUP_{S\sigma}(A\sigma, L') \neq -\infty$. Then $SUP_{S\sigma}(A\sigma, L') = [SUP_S(A, L)]\sigma$.*

To see that this implies Theorem 13 we just let $\sigma = \{v/r\}$, $L' = \emptyset$, $L = \{v\}$, and $T = S\sigma$. Note that if T is satisfiable it follows from the conservativeness of SUP that $SUP_{S\sigma}(A, \emptyset) \neq -\infty$.

The proof is as usual, by recursion induction. All nontrivial cases reduce to the one in which A is a variable not in L (hence $A\sigma = A$):

$$(1) SUP_{S\sigma}(A\sigma, L') = SUP_{S\sigma}(A, L'),$$

$$(2) = SUPP(A, SUP_{S\sigma}(\text{UPPER}_{S\sigma}(A), L' \cup \{A\}))$$

Now since $SUP_{S\sigma}(A, L') \neq -\infty$, it must be (by inspection of $SUPP$) that $SUP_{S\sigma}(\text{UPPER} \dots) \neq -\infty$. By induction hypothesis, then,

$$SUP_{S\sigma}(\text{UPPER}_{S\sigma}(A), L' \cup \{A\}) = SUP_S(\text{UPPER}_S(A), L \cup \{A\})\sigma.$$

(Note that $L \cup \{A\}$ plays the role of L and $L' \cup \{A\}$ plays that of L' ; note also that $\text{UPPER}_{S\sigma}(A) = \text{UPPER}_S(A)\sigma$.)

Thus (2) above becomes

$$(3) SUPP(A, SUP_S(\text{UPPER}_S(A), L \cup \{A\})\sigma).$$

Now, inspection of $SUPP$ will show that for any minilinear Q , if $SUPP(A, Q\sigma) \neq -\infty$, $SUPP(A, Q\sigma) = SUPP(A, Q)\sigma$. Thus, (3) becomes

(4) $\text{SUPP}(A, \text{SUP}_S(\text{UPPER}_S(A), L \cup \{A\}))\sigma$,

(5) $= \text{SUP}_S(A, L)\sigma$

as desired. $\square$

12. The Main Theorem

We now have all the equipment necessary to prove the main result.

THEOREM 17. *If $v \in V$ and S bounds v , $\text{SUP}_S(v, \emptyset) = \max_S v$.*

PROOF. The proof is by induction on the number n of distinct variables occurring in inequalities in S . In order to push the induction through, we will actually need to prove a somewhat stronger result:

(i) If $v \in V$ and S bounds v , $\text{SUP}_S(v, \emptyset) = \max_S v$.

(ii) If L is a linear form with a unique value r in S , $\text{SUP}_S(L, \emptyset) = r$.

It will also be necessary to carry the corresponding results for INF as part of the induction hypothesis. Since the argument for INF is the exact dual of the one for SUP, it will not be given explicitly.

Basis. $n = 0$.

In this case S has no variables appearing. Part (i) must therefore hold vacuously. Part (ii) holds since L must be the constant r ; hence $\text{SUP}_S(L, \emptyset) = r$.

Induction step. $n > 0$.

Part (i). Let v_M denote $\max_S v$. We need to show that $v_M = \text{SUP}_S(v, \emptyset)$. By Theorem 8, there exists a subset S' of S such that S' minimally bounds v and $\max_{S'} v = v_M$. Now we know by Theorem 11 that $v_M \leq \text{SUP}_S(v, \emptyset)$. Furthermore, since $S' \subseteq S$, we know from Theorem 13 that $\text{SUP}_S(v, \emptyset) \leq \text{SUP}_{S'}(v, \emptyset)$. Hence, $v_M \leq \text{SUP}_S(v, \emptyset) \leq \text{SUP}_{S'}(v, \emptyset)$, and so it will suffice to show that $\text{SUP}_{S'}(v, \emptyset) = v_M$.

Now by definition of SUP,

$$\text{SUP}_{S'}(v, \emptyset) = \text{SUPP}(v, \text{SIMP}(Z)),$$

where $Z = \text{SUP}_{S'}(Q, \{v\})$ and $Q = \text{UPPER}_{S'}(v)$.

Note that by Theorem 12, Z can be considered as a function of v . Our plan is to show that $Z(v_M) = v_M$ and that for some $v' < v_M$, $Z(v') > v'$. It will then follow from Theorem 14 that $\text{SUPP}(v, \text{SIMP}(Z)) = v_M$, and hence $\text{SUP}_{S'}(v, \emptyset) = v_M$.

So let T be obtained from S' by substituting v_M everywhere for v . Then to show that Z has a fixed point at v_M , it will suffice, by Theorem 15, to show that $\text{SUP}_T(Q, \emptyset) = v_M$. Now Q is expressible either as L_1 or as $\text{MIN}(L_1, L_2, \dots, L_n)$, where for $1 \leq i \leq n$, $v \leq L_i$ is an inequality in S' . We will consider only the latter case; the former is handled similarly. By Theorem 9, each L_i has the unique value v_M in T , and Q therefore has that value uniquely as well. Then

$$\begin{aligned} \text{SUP}_T(Q, \emptyset) &= \text{SUP}_T(\text{MIN}(L_1, L_2, \dots, L_n)) \\ &= \text{MIN}(\text{SUP}_T(L_1, \emptyset), \text{SUP}_T(L_2, \emptyset), \dots, \text{SUP}_T(L_n, \emptyset)) \\ &= \text{MIN}(\max_T L_1, \dots, \max_T L_n) \quad (\text{by induction hypothesis, part (ii)}) \\ &= \max_T Q \quad (\text{by Theorem 1}) \\ &= v_M. \end{aligned}$$

Thus v_M is a fixed point of Z .

It remains to show that for some $v' < v_M$, $Z(v') > v'$. Once again writing Q as $\text{MIN}(L_1, L_2, \dots, L_n)$, it follows from Theorem 10 that there exists a $v' < v_M$ such that if T is obtained from S' by substituting v' everywhere for v , $\max_T L_i$ is defined and is greater than v' for each i , $1 \leq i \leq n$. Then

$$\begin{aligned} Z(v') &= \text{SUP}_T(Q, \emptyset) \quad (\text{by Theorem 15}) \\ &= \text{SUP}_T(\text{MIN}(L_1, \dots, L_n), \emptyset) \\ &= \text{MIN}(\text{SUP}_T(L_1, \emptyset), \dots, \text{SUP}_T(L_n, \emptyset)) \\ &\geq \text{MIN}(\max_T L_1, \dots, \max_T L_n) \quad (\text{by Theorem 11}) \\ &> \text{MIN}(v', \dots, v') = v' \end{aligned}$$

as required.

Part (ii). We need to show that if L is a linear form with a unique value r in S , $\text{SUP}_S(L, \emptyset) = r$. We distinguish two cases:

Case 1. L has at most one variable appearing. In this case L can be expressed $bv + c$ where $v \in V$ and b and c are reals. Now since L has the unique value r in S , either $b = 0$ and $c = r$ in which case $\text{SUP}_S(L, \emptyset) = r$ trivially, or v has the unique value $(r - c)/b$ in S . In this latter case,

$$\max_S v = \min_S v = (r - c)/b$$

and so

$$\begin{aligned} \text{SUP}_S(L, \emptyset) &= \begin{cases} b \text{ SUP}_S(v, \emptyset) + c & \text{if } b > 0 \\ b \text{ INF}_S(v, \emptyset) + c & \text{if } b < 0 \end{cases} \\ &= \begin{cases} b \max_S v + c & \text{if } b > 0 \\ b \min_S v + c & \text{if } b < 0 \end{cases} \text{ by the argument of part (i)} \\ &= b(r - c)/b + c = r \text{ as needed.} \end{aligned}$$

Case 2. L has more than one variable appearing. In this case L can be expressed $bv + L'$ where b is a nonzero real, v is a variable not occurring in L' , and L' is a linear form. Letting $Q = \text{SUP}_S(L', \{v\})$, we have:

$$\text{SUP}_S(L, \emptyset) = \begin{cases} \text{SUP}_S(\text{SIMP}(bv + Q), \emptyset) & \text{if } v \text{ occurs in } Q, \\ \text{SUP}_S(bv, \emptyset) + Q & \text{otherwise.} \end{cases}$$

Now suppose v' is a value that v can take in S , and let T be obtained from S by substituting v' everywhere for v . By Theorem 15, if Q is considered as a function of v , $Q(v') = \text{SUP}_T(L', \emptyset)$. Now since L has the unique value r in S , L' must have the unique value $r - bv'$ in T . By induction hypothesis, then, $Q(v') = \text{SUP}_T(L', \emptyset) = r - bv'$. The value $Q(v')$ is thus the one and only value that L' can have in S when v is fixed at v' . $bv' + Q(v')$ is thus the one and only value L can have in S when v is fixed at v' . But since L has the same value r in S regardless of where v is fixed, either v has a unique value in S or $\text{SIMP}(bv + Q)$ is the constant r . In either case, $bv + Q$ has the unique value r in S .

Now if v does not occur in Q , $\text{SUP}_S(L, \emptyset) = \text{SUP}_S(bv, \emptyset) + Q$. By the argument of part (i), $\text{SUP}_S(bv, \emptyset) = \max_S(bv)$. Furthermore, since Q must be a constant in this case,

$$\text{SUP}_S(L, \emptyset) = \max_S(bv) + Q = \max_S(bv + Q) = r.$$

In the remaining case, v does occur in Q , and $\text{SUP}_S(L, \emptyset) = \text{SUP}_S(\text{SIMP}(bv + Q), \emptyset)$. If $\text{SIMP}(bv + Q)$ is the constant r , the result follows immediately. Otherwise, as was noted above, v has a unique value in S . Now either $\text{SIMP}(bv + Q)$ is linear in $\{v\}$ in which case $\text{SUP}_S(L, \emptyset) = \text{SUP}_S(\text{SIMP}(bv + Q), \emptyset) = \max_S(bv + Q) = r$ by the argument of case 1, or $\text{SIMP}(bv + Q)$ can be written $\text{MIN}(L_1, \dots, L_N)$ where each L_i is linear in $\{v\}$. In this latter case,

$$\begin{aligned} \text{SUP}_S(L, \emptyset) &= \text{MIN}(\text{SUP}_S(L_1, \emptyset), \dots, \text{SUP}_S(L_N, \emptyset)) \\ &= \text{MIN}(\max_S L_1, \max_S L_2, \dots, \max_S L_N) \text{ (by the argument of case 1)} \\ &= \max_S \text{MIN}(L_1, L_2, \dots, L_N) \text{ (by Theorem 1)} \\ &= \max_S(bv + Q) = r \end{aligned}$$

as required. $\square$

ACKNOWLEDGMENTS. The author is grateful to W.W. Bledsoe, D.C. Cooper, Robert Hill, R. Lipton, and M.W. Green for their suggestions and patient readings of the proof.

REFERENCES

(Note. References [5, 7] are not cited in the text)

- 1 BLEDSOE, W W The SUP-INF method in Presburger arithmetic Memo ATP-18, Math Dept , U of Texas at Austin, Austin, Tex , Dec 1974.

- 2 BLEDSOE, W W A new method for proving certain Presburger formulas Advance Papers 4th Int Joint Conf on Artif Intell, Tbilisi, Georgia, U S S R, Sept 1975, pp 15-21
- 3 COOPER, D C Programs for mechanical program verification In *Mach Intell* 6, B Meltzer and D Michie, Eds., American Elsevier, New York, 1971, pp 43-59
- 4 COOPER, D C Theorem proving in arithmetic without multiplication In *Mach. Intell* 7, B. Meltzer and D Michie, Eds, American Elsevier, New York, 1972, pp 91-99
- 5 DAVIS, M A program for Presburger's algorithm Summer Inst for Symbolic Logic, Cornell U, Ithaca, N Y, 1957, pp 215-233
- 6 KREISEL, G, AND KREVINE, J L *Elements of Mathematical Logic* North-Holland, Amsterdam, 1967, pp 54-57
- 7 LEE, R D An application of mathematical logic to the integer linear programming problem *Notre Dame J Formal Logic XIII*, 2 (April 1972), 279-282
- 8 OPPEN, D Ph D Th, U. of Toronto, Toronto, Ont, 1975, *J Compr Syst Sci* (to appear).
- 9 PRESBURGER, M Uber die Vollständigkeit eines gewissen Systems der Arithmetic ganzer Zahlen In *Welchem die Addition als Einzige Operation hervortritt*, Sprawozdanie z I Kongresu Matematyków Krajow Slowcanskich Warszawa, 1929, pp 92-101

RECEIVED OCTOBER 1975, REVISED MARCH 1977